Ferbin Richard

Robotics Software Engineer, Autonomous Underwater Systems

Engineering Brag Document · Archenaux Robotics (formerly Airnaux) · Co-op, 31 May 2026 to present · Compiled 30 July 2026

> **Scope of this record.** This document describes work I performed, the engineering decisions I made, and outcomes measured against defined baselines. It is written at the level of published capability and contains no source code, proprietary designs, customer information, or internal datasets. All figures are reproduced from evaluation reports I authored and from version-control history.

220	~41k	5	9
COMMITS AUTHORED	LINES OF CODE	REPOS OWNED	WEEKS ELAPSED

Summary

I build the simulation, perception, and deployment infrastructure for an autonomous underwater inspection vehicle. Over nine weeks I took the program from an empty repository to a working stack spanning physics simulation, factor-graph SLAM, learned mine detection, and flight-verified firmware running on real hardware in the water.

Three themes run through the work. First, **replacing expensive hardware with algorithms**: I helped validate a localization approach that removes an $8,710 Doppler Velocity Log from the sensor budget without sacrificing map accuracy. Second, **making the stack reproducible for other people**: much of my commit history is portability, one-command bring-up, and documentation, because a simulator only one engineer can run is not an asset. Third, **measuring honestly**: I built the evaluation harnesses that produced the numbers below, including the ones that show where the models still fail.

Technical Environment

Area	Tools and platforms
Robotics	ROS 2 Jazzy, Gazebo Harmonic, ArduPilot/ArduSub, MAVLink, MAVROS, QGroundControl, RViz2, NVIDIA Isaac Sim
Perception / SLAM	GTSAM (iSAM2), factor-graph optimization, loop closure, GICP registration, Ultralytics YOLO11 / YOLO26, OpenCV, point-cloud processing
Languages	Python, C++, Bash, SDF/URDF, YAML
Infrastructure	Docker, Docker Compose, VSCode dev containers, GitHub Actions CI, CycloneDDS, Xvfb/noVNC, EGL headless rendering
Hardware	Pixhawk flight controllers, NVIDIA Jetson, Raspberry Pi 5, Ping360 scanning sonar, Intel RealSense D455, VN-100 IMU, Bar30 depth sensor, brushless thruster ESCs

Project 1 · DVL-Free Hull Mapping and SLAM

Primary engineer, sensor simulation and SLAM back-end · Jun 2 to Jul 27, 2026 · 45 commits, ~8,800 LOC

Problem. The vehicle needed to produce a metrically accurate coloured 3D map of a ship hull. The industry-standard solution is a Doppler Velocity Log, at $8,710 the single most expensive sensor in the design, roughly four times the sonar and seventeen times the IMU.

Thesis I helped validate. A DVL is not required. An IMU, a 2D mechanical scanning sonar, the known perimeter and contour of the structure being inspected, and loop closure together yield a clean, metrically accurate map.

What I built.

- An eleven-stage mapping pipeline from raw sonar returns to an optimized coloured point cloud, with a GTSAM iSAM2 factor-graph back-end, loop-closure detection, and GICP-based registration evaluated against simulation ground truth.
- A physically faithful Ping360 sensor model. The sonar is a rotating single-beam device with no elevation resolution, so a naive 3D implementation produces a plausible-looking but physically wrong point cloud. I corrected the output to a true flat 2D disc, fixed a 180-degree disc flip, and added a free-surface cull so the air-water interface stopped injecting phantom returns.
- A two-phase colour mapping mode fusing Intel RealSense D455 imagery onto the sonar-derived geometry, plus an automated coverage probe and reproducible report builder with seeded RNG so published figures are exactly reproducible.

Notable debugging. The SLAM back-end diverged whenever a loop closure fired, which is the failure mode that makes a factor graph worse than dead reckoning. Separately, the registration acceptance threshold shipped at a default of 0.15, which admitted bad matches and produced drift out of the box; raising it to 0.6 after characterizing the score distribution made the pipeline drift-free without tuning.

Metric, Bremen MSS real-pool dataset	Result	Baseline
Absolute trajectory error, DVL-free localization	**1.62 m**	5.04 m (open-loop DVL dead reckoning)
Wall RMSE	**0.08 m**	—
Hardware cost removed from sensor budget	**$8,710**	Water Linked A50 DVL

DVL-free localization outperformed the dead-reckoning baseline it was designed to replace, on real-world data rather than simulation.

Project 2 · Mine Detection: Simulation, Dataset, and Model

Sole engineer, Phase 1 simulation and ML pipeline · Jul 7 to Jul 14, 2026 · 26 commits, ~7,800 LOC

Problem. Build a mine countermeasures capability with no access to real minefield imagery, which is classified or nonexistent. I owned Phase 1 end to end: generate the data, train a detector, evaluate it honestly, and hand a documented interface to the ML team.

What I built.

- A custom three-camera plus sonar vehicle rig and a seeded procedural minefield world generator emitting ground-truth labels by 3D-to-2D projection, so every frame is labeled for free and any scenario is reproducible from its seed.
- Deliberate difficulty and domain randomization: seven mine types, three turbidity levels, ocean-current drift, marine snow, kelp, fish schools, and seabed distractors including barrels, anchors, crates, and tyres that a naive detector will call mines.
- An evaluation harness sweeping mine type against distance, angle, and turbidity, reporting mAP, per-class precision and recall, false-alarm rate on hard negatives, and a confidence-threshold sweep for operational threshold selection.
- Iterative model deployment through `mine_yolo11_v3.pt` and a YOLO26s detector with an added far-distance evaluation tier, plus a data-format specification and offline-first deployment strategy for a vehicle with no network link.

Results. The clutter false-positive rate went from **54% to 0%** between model revisions, which was the blocking problem: a detector that flags every barrel as a mine is operationally useless regardless of its recall.

Evaluation, `mine_yolo11.pt` on rendered sim sweep	Result
False alarms on empty background frames	**0% (0/9)**
Recall, moored contact and drifting mines	**100% (15/15 each)**
mAP@50, simulation	0.471
mAP@50, real-photo training domain	0.61
Recall, top-down vs forward camera geometry	57% vs 39%

What the evaluation revealed. Recall was 100% on spherical and horned mines and 0% on manta, CAPTOR, and limpet types, because the real-photo training set contained no flat or torpedo-shaped mines. The sim-to-real gap of 0.471 against 0.61 quantified the domain shift. I documented both findings and identified fine-tuning on the labeled sim data as the highest-value next step, which is precisely what the dataset was built to enable. I consider surfacing these gaps a deliverable, not a shortfall: an unevaluated detector deployed to a minefield is a safety problem.

Project 3 · Vehicle Firmware and Field Deployment

Primary engineer, containerized runtime and hardware bring-up · Jun 12 to Jul 27, 2026 · 52 commits, ~12,500 LOC

This is the work that runs on the physical vehicle, where simulator bugs become recovery dives.

- **Containerized runtime** for the real vehicle: MAVROS companion computer and topside control via Docker Compose profiles, with a single `run.sh` front door. Added an `onboard` command taking a blank laptop to a running stack in one invocation, which matters when you are setting up at a test site rather than a desk.
- **Firmware release engineering:** a three-build manifest covering default, asymmetric, and per-operator configurations, over-Ethernet flashing with automatic parameter backup, and dataflash log retrieval. All three builds were flash-verified on hardware ahead of the pool test.
- **Portability fixes** found the hard way. DDS discovery bound to interface names and broke on every new laptop, so I rebound it by IP address. The Pixhawk enumerated at unstable device paths, so I added a stable `/dev/pixhawk` rule. QGroundControl configuration failed to seed on fresh clones, a bug invisible on any machine that had already run once.
- **Sensor bring-up:** Ping360 sonar and VN-100 IMU with live RViz visualization. I traced an x86-only scan-parsing failure to an uninitialized `PingParser` buffer length, and fixed sector angles that wrapped and broke monotonic LaserScan output.
- **FPV goggles OSD** driven by live MAVLink with dual-branch video streaming, extended with an RTP video mode so the same pilot hardware drives the simulator.

Project 4 · Simulation Platform and Developer Infrastructure

Owner, shared simulation core · May 31 to Jul 29, 2026 · 97 commits across two repos, ~11,800 LOC

I own the simulation core the other projects build on. Its value is that other engineers can actually run it.

- **Full underwater stack:** ArduSub SITL wired to Gazebo Harmonic over a JSON FDM backend, an eight-thruster vectored 6-DOF vehicle matching real hardware, QGroundControl with joystick passthrough, and a compose-based VSCode dev container. Validated across three separate laptops.
- **Runs anywhere.** GPU made optional with auto-detected NVIDIA and DRI overrides, WSL2/WSLg support, and a browser-based Xvfb plus noVNC mode so the GUI works on headless, Wayland, and WSL hosts. This directly resolved a display crash blocking a teammate.
- **Reliability engineering:** a QA harness with scenario self-tests and built-image content checks, catching the class of failure where an image is missing a tool. Scripts self-heal stale Docker images, X11 auth cookies, and permission errors instead of failing with cryptic mount messages.
- **NVIDIA Isaac Sim port** (July 29): Archelon vehicle and Fort Lauderdale marina scene, an animated Gerstner-wave ocean with underwater rendering, underwater dynamics with buoyancy, damping, and added mass, an eight-thruster model, and an ArduSub JSON FDM bridge closing the SITL loop against Isaac physics. I wrote an adversarial QA suite and used mutation testing to tighten the hydrodynamics assertions, which surfaced distributed-buoyancy and NaN-propagation bugs before they reached anyone else.
- **Architecture:** authored the proposal establishing `archenaux_sim` as a canonical core consumed via overlays rather than per-team forks, preventing the divergence that had already begun.

Engineering Practices

- **Documentation as deliverable.** ~6,200 lines of committed documentation: teammate-facing run guides, architecture diagrams, interface contracts, a handoff document for the ML team, electrical and

power-wiring references including a Jetson voltage-sag analysis, and a pool-test checklist.

- **Testing.** SITL regression tests for firmware behaviour, unit-verified pipeline stages, adversarial and mutation testing on the physics model, and Tier-1 GitHub Actions CI with PR and issue templates.
- **Reproducibility.** Seeded RNG for published figures, committed evaluation artifacts, and replication guides specifying required hardware.
- **Enabling teammates.** A recurring pattern in the commit history: onboarding READMEs, one-command setup, and fixing the specific machine-dependent failures other engineers hit.

Interview Talking Points

If asked about	Lead with
Hard technical problem	The loop closure that exploded the factor graph, and the registration threshold that made the SLAM pipeline drift-free by default rather than after tuning.
Working with ambiguity	Building a mine detector with no real minefield data: procedural generation with free ground-truth labels and deliberate hard negatives.
Engineering judgment	Modeling the Ping360 as a true 2D disc. The 3D version looked better in RViz and was physically wrong.
Impact	Validating DVL-free localization: $8,710 removed from the sensor budget, better trajectory error than the baseline it replaced.
Owning failure	The detector evaluation I ran, which showed 0% recall on three mine types, and how I turned that into the fine-tuning plan.
Teamwork	Portability and QA work: the simulator was only useful once other engineers could run it on their own machines.

Compiled 30 July 2026 from version-control history and evaluation reports authored by Ferbin Richard. Contains no proprietary source, designs, or data. Figures are reproducible from the cited evaluation runs. Maintained as an ongoing personal record, updated monthly.